

NBA 파생지표 & 생성 코드 템플릿

기준 데이터: games.csv, games_details_shortver.csv(또는 game_details 원본), ranking_stver.csv, players.csv, teams.csv | 작성일: 2026-01-30

1. 무엇을 만들 수 있나

이 문서는 (1) 선수-경기 단위 박스스코어에서 효율/스타일 지표를 만들고, (2) 팀-경기 단위 지표(홈/원정, 득실점차 등)를 만들며, (3) 팀-시즌 단위 결과(ranking)의 승률/홈-원정 의존도와 결합하는 흐름을 제공합니다. 분석 목적에 따라 필요한 지표만 골라 사용하세요.

2. 원천 지표 요약

2.1 games_details (선수-경기)

범주	컬럼
출전	MIN (문자열 mm:ss)
득점/슛	PTS, FGM, FGA, FG_PCT, FG3M, FG3A, FG3_PCT, FTM, FTA, FT_PCT
리바운드	OREB, DREB, REB
플레이메이킹	AST
수비	STL, BLK
실수/파울	TO, PF
임팩트	PLUS_MINUS
기타	START_POSITION(있을 수 있음), TEAM_ID/PLAYER_ID/NAME 등

2.2 games (팀-경기)

범주	컬럼
득점	PTS_home, PTS_away
슛 효율	FG_PCT_home/away, FG3_PCT_home/away, FT_PCT_home/away
팀 플레이	AST_home/away
리바운드	REB_home/away
결과	HOME_TEAM_WINS(0/1), GAME_DATE_EST, HOME_TEAM_ID, VISITOR_TEAM_ID

2.3 ranking (팀-시즌)

범주	컬럼
시즌 결과	W, L, W_PCT
홈/원정	HOME_RECORD, ROAD_RECORD
키	TEAM_ID, SEASON_ID(또는 시즌 관련 컬럼)

3. 파생 지표 정의

3.1 선수-경기 파생 지표 (games_details)

지표	설명	공식
MIN_float	MIN(mm:ss)을 분 단위 float로 변환	$MIN_float = mm + ss/60$
PTS_per36	분당 득점 보정	$PTS_per36 = PTS / MIN_float * 36$
REB_per36	분당 리바운드 보정	$REB_per36 = REB / MIN_float * 36$
AST_per36	분당 어시스트 보정	$AST_per36 = AST / MIN_float * 36$
STL_per36	분당 스틸 보정	$STL_per36 = STL / MIN_float * 36$
BLK_per36	분당 블록 보정	$BLK_per36 = BLK / MIN_float * 36$
TO_per36	분당 턴오버 보정	$TO_per36 = TO / MIN_float * 36$
eFG_pct	2점+3점 가중 슛 효율	$eFG\% = (FGM + 0.5 * FG3M) / FGA$
TS_pct	자유투 포함 득점 효율	$TS\% = PTS / (2 * (FGA + 0.44 * FTA))$
FG3A_rate	3점 시도 비중	$FG3A_rate = FG3A / FGA$
FTr	자유투 시도 비중	$FTr = FTA / FGA$
AST_TO	어시스트/턴오버	$AST_TO = AST / TO$ (TO=0이면 NaN 처리)
OREB_share	공격리바운드 비중	$OREB_share = OREB / REB$
Impact_basic	간단 합성 임팩트	$PTS + REB + AST + STL + BLK - TO$
USG_proxy_per_min	사용량 proxy(분당)	$(FGA + 0.44 * FTA + TO) / MIN_float$

3.2 팀-경기 파생 지표 (games)

지표	설명	공식
Point_diff	홈-원정 득점차	$PTS_home - PTS_away$
AST_diff	홈-원정 어시스트차	$AST_home - AST_away$
REB_diff	홈-원정 리바운드차	$REB_home - REB_away$
FG_pct_diff	슛 성공률 격차	$FG_PCT_home - FG_PCT_away$
FG3_pct_diff	3점 성공률 격차	$FG3_PCT_home - FG3_PCT_away$
FT_pct_diff	자유투 성공률 격차	$FT_PCT_home - FT_PCT_away$
Total_points	총 득점	$PTS_home + PTS_away$
Abs_point_diff	점전 정도	$abs(PTS_home - PTS_away)$

3.3 팀-시즌 파생 지표 (ranking)

지표	설명	공식
HOME_WIN_PCT	홈 승률	$home_wins / (home_wins + home_losses)$
ROAD_WIN_PCT	원정 승률	$road_wins / (road_wins + road_losses)$
HOME_DEPENDENCY	홈 의존도	$HOME_WIN_PCT - ROAD_WIN_PCT$
NET_WINS	순승	$W - L$

4. 파생지표 전부 생성 코드 템플릿

아래 코드는 (A) 파일 로드, (B) MIN 파싱, (C) 선수-경기 파생지표 생성, (D) 팀-경기 파생지표 생성, (E) ranking 파생지표 생성, (F) 조인까지 한 번에 수행합니다. 데이터에 따라 컬럼명이 조금 다를 수 있으니, 파일을 바꾸면 먼저 df.columns를 확인하세요.

```
# =====
# NBA Derived Metrics Template
# =====

import pandas as pd
import numpy as np

# 0) 파일 경로 (경로 지정)
PATH_GAMES = "games.csv"
PATH_DETAILS = "games_details_shortver.csv" # 파일 이름 game_details.csv
PATH_RANKING = "ranking_stver.csv"
PATH_PLAYERS = "players.csv"
PATH_TEAMS = "teams.csv"

# 1) 파일 로드
games = pd.read_csv(PATH_GAMES)
details = pd.read_csv(PATH_DETAILS)
ranking = pd.read_csv(PATH_RANKING)
players = pd.read_csv(PATH_PLAYERS)
teams = pd.read_csv(PATH_TEAMS)

# 2) MIN: MIN(mm:ss) -> float(float)
def parse_min_to_float(x):
    if pd.isna(x):
        return np.nan
    if isinstance(x, (int, float)):
        return float(x)
    s = str(x).strip()
    if s in {"", "0", "0:00", "0:0"}:
        return 0.0
    if ":" in s:
        mm, ss = s.split(":")
        try:
            return float(mm) + float(ss)/60.0
        except:
            return np.nan
    try:
        return float(s)
    except:
        return np.nan

# 3) 안전한 나눗셈
def safe_div(a, b):
    a = np.asarray(a, dtype="float64")
    b = np.asarray(b, dtype="float64")
    return np.where((b == 0) | np.isnan(b), np.nan, a / b)

# 4) details 테이블에 MIN 파싱 적용
details["MIN_float"] = details["MIN"].apply(parse_min_to_float)

FGA = details["FGA"].to_numpy()
FGM = details["FGM"].to_numpy()
FG3M = details["FG3M"].to_numpy()
FG3A = details["FG3A"].to_numpy()
FTA = details["FTA"].to_numpy()
PTS = details["PTS"].to_numpy()
MINf = details["MIN_float"].to_numpy()
T0 = details["T0"].to_numpy()
REB = details["REB"].to_numpy()
OREB = details["OREB"].to_numpy()

# PTS per 36 계산
details["PTS_per36"] = safe_div(details["PTS"], details["MIN_float"]) * 36
```

```

details["REB_per36"] = safe_div(details["REB"], details["MIN_float"]) * 36
details["AST_per36"] = safe_div(details["AST"], details["MIN_float"]) * 36
details["STL_per36"] = safe_div(details["STL"], details["MIN_float"]) * 36
details["BLK_per36"] = safe_div(details["BLK"], details["MIN_float"]) * 36
details["TO_per36"] = safe_div(details["TO"], details["MIN_float"]) * 36

# []
details["eFG_pct"] = safe_div(FGM + 0.5*FG3M, FGA)
details["TS_pct"] = safe_div(PTS, 2*(FGA + 0.44*FTA))
details["FG3A_rate"] = safe_div(FG3A, FGA)
details["FTr"] = safe_div(FTA, FGA)

# []/[]
details["AST_TO"] = safe_div(details["AST"].to_numpy(), TO)
details["OREB_share"] = safe_div(OREB, REB)
details["Impact_basic"] = (
    details["PTS"] + details["REB"] + details["AST"] + details["STL"] + details["BLK"] - details["TO"]
)
details["USG_proxy_per_min"] = safe_div(FGA + 0.44*FTA + TO, MINf)

# 5) games []
games["Point_diff"] = games["PTS_home"] - games["PTS_away"]
games["AST_diff"] = games["AST_home"] - games["AST_away"]
games["REB_diff"] = games["REB_home"] - games["REB_away"]
games["FG_pct_diff"] = games["FG_PCT_home"] - games["FG_PCT_away"]
games["FG3_pct_diff"] = games["FG3_PCT_home"] - games["FG3_PCT_away"]
games["FT_pct_diff"] = games["FT_PCT_home"] - games["FT_PCT_away"]
games["Total_points"] = games["PTS_home"] + games["PTS_away"]
games["Abs_point_diff"] = (games["PTS_home"] - games["PTS_away"]).abs()

# 6) ranking [] (HOME_RECORD, ROAD_RECORD: "W-L" [] [])
def parse_record(rec):
    if pd.isna(rec):
        return (np.nan, np.nan)
    s = str(rec).strip()
    if "-" not in s:
        return (np.nan, np.nan)
    w, l = s.split("-")
    try:
        return (int(w), int(l))
    except:
        return (np.nan, np.nan)

ranking[["home_wins", "home_losses"]] = ranking["HOME_RECORD"].apply(lambda x: pd.Series(parse_record(x)))
ranking[["road_wins", "road_losses"]] = ranking["ROAD_RECORD"].apply(lambda x: pd.Series(parse_record(x)))

ranking["HOME_WIN_PCT"] = safe_div(ranking["home_wins"], ranking["home_wins"] + ranking["home_losses"])
ranking["ROAD_WIN_PCT"] = safe_div(ranking["road_wins"], ranking["road_wins"] + ranking["road_losses"])
ranking["HOME_DEPENDENCY"] = ranking["HOME_WIN_PCT"] - ranking["ROAD_WIN_PCT"]
ranking["NET_WINS"] = ranking["W"] - ranking["L"]

# 7) ([]) [] -> [] [] (TEAM_ID []; [] [] [] [])
team_player_summary = (
    details.groupby("TEAM_ID")
    .agg(
        team_avg_TS=("TS_pct", "mean"),
        team_avg_eFG=("eFG_pct", "mean"),
        team_avg_PTS36=("PTS_per36", "mean"),
        team_avg_ASTTO=("AST_TO", "mean"),
        team_avg_Impact=("Impact_basic", "mean"),
        players=("PLAYER_ID", "nunique")
    )
    .reset_index()
)

# 8) ranking[] [] + [] [] []
rank_enriched = ranking.merge(team_player_summary, on="TEAM_ID", how="left")
rank_enriched = rank_enriched.merge(teams, on="TEAM_ID", how="left")

# 9) []

```

```
details.to_csv("details_with_derived_metrics.csv", index=False)
games.to_csv("games_with_derived_metrics.csv", index=False)
rank_enriched.to_csv("ranking_enriched_with_player_metrics.csv", index=False)

print("Saved:")
print("- details_with_derived_metrics.csv")
print("- games_with_derived_metrics.csv")
print("- ranking_enriched_with_player_metrics.csv")
```

5. 사용 팁

- 대용량 game_details.csv는 chunksize로 읽고 필요한 컬럼만(usecols) 가져오면 훨씬 빠릅니다.
- per36는 MIN_float가 너무 작은 경우 왜곡됩니다. 보통 MIN_float>=5 또는 >=10 필터를 추천합니다.
- TS%, eFG%는 FGA/FTA가 0이면 NaN이 됩니다. 분석 목적에 따라 0 처리 또는 제외를 선택하세요.
- 팀-시즌 결합은 가능한 경우 시즌 키(SEASON 또는 SEASON_ID)를 함께 사용하세요. TEAM_ID만 쓰면 시즌 혼합 위험이 있습니다.